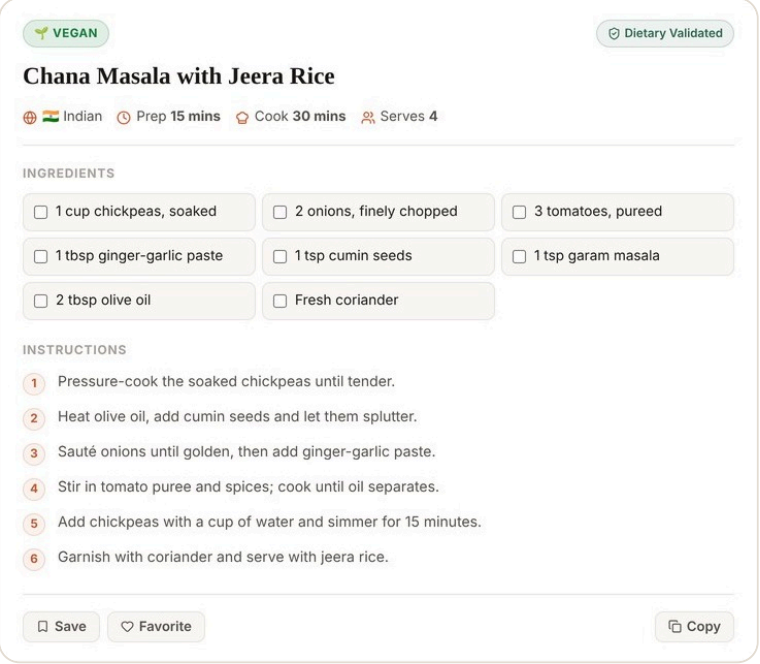
PROJECT REPORT · BUILT FOR VELTRIS

Nourish SLM: a personalized food and diet assistant using a fine-tuned small language model

Recipes from the ingredients you have, checked against your diet and allergies, with nutrition per serving and full-day diet plans for 24 goals and sports.

Author	KM Sreenidhi, CSE (AI & ML), Woxsen University
Organisation	Veltris
Date	10 November 2026
Code	github.com/sreenidhi-km/nourish-slm



−62% test loss after fine-tuning (2.895 → 1.106)

97% complete, well-formed recipes (was 66.7%)

24 diet plans from 46,514 real recipes

73 automated tests passing

Abstract

General-purpose chatbots suggest recipes without checking them: they put ghee in a vegan dish, ignore allergies, give no nutrition facts and send personal data to paid cloud APIs. Nourish SLM is a self-hosted web application that addresses these gaps. It uses Qwen3-4B, an open-weight small language model, fine-tuned with QLoRA on 6,000 instruction examples drawn from 229,168 cleaned Food.com recipes. Every answer then passes through deterministic checks: a dietary validator, an allergy checkpoint covering 10 allergens, and a nutrition engine based on USDA FoodData Central.

A Diet section adds 24 plans (dietary preferences, fitness goals and 12 sports) built from 46,514 recipes with photos and published nutrition. It provides personal targets, a Check My Diet review and a plan-specific assistant. Fine-tuning lowered test loss by 62% and raised the share of complete recipes from 66.7% to 97.0%, while dietary compliance stayed at 100%. The validator passes 50 of 50 test cases at about 0.1 ms per check, and the full suite of 73 automated tests passes. The system runs on a MacBook, a GPU server or Docker, with no external AI API.

Contents

1. Introduction and objectives
2. The product
3. System architecture
4. Data
5. Model and fine-tuning
6. Dietary safety layer
7. Allergy checkpoint
8. Nutrition engine
9. Diet planning
10. Backend and frontend
11. Evaluation
12. Deployment
13. Limitations and future work
14. References

1 Introduction and objectives

People increasingly ask chatbots what to cook. Generic assistants answer fluently, but four problems recur:

- **No safety checks.** Diet rules and allergies are not enforced. A vegan can be offered paneer butter masala finished with ghee.
- **No nutrition.** Calories and macronutrients are guessed or missing.
- **No plan.** Suggestions are one-off. Nothing is built around a person's goal or training schedule.
- **Cost and privacy.** Cloud APIs are paid per request and see the user's data.

The project set out to:

1. clean the Food.com dataset and fine-tune Qwen3-4B to write structured recipes;
2. validate every recipe against the user's diet and allergies;

3. show nutrition per serving on every recipe;
4. build personalised diet plans for 24 goals and sports;
5. measurably beat the base model;
6. run fully on the owner's hardware.

2 The product

A user signs in, picks a dietary preference (Vegan, Vegetarian, Eggitarian, Non-vegetarian or No preference) and, optionally, any of 10 allergies. They then describe their kitchen in plain language, for example "I have chickpeas, onions and tomatoes. Make me something Indian." Nourish replies with a structured recipe card. The card shows:

- title, diet badge, cuisine, prep and cook time, and servings;
- checkable ingredients, now with suggested amounts;
- numbered steps;
- a *Dietary validated* badge and an allergy check panel;
- nutrition per serving.

The diet and allergy settings live in the header and can be changed at any time. The app also has:

- chat history, saved recipes and favourites;
- light and dark themes;
- a **Diet** section with 24 plans, each with its own day of meals, review and assistant.

3 System architecture



Figure 1. Request flow. The model writes the recipe; the diet validator, allergy checkpoint and nutrition engine decide what the user sees.

A request goes through these steps:

1. The React app sends it to the FastAPI backend.

2. The backend attaches the user's diet, cuisine and allergies.
3. It retrieves similar recipes from 229,168 indexed recipes.
4. The fine-tuned model writes a JSON recipe.
5. Three deterministic stages run in order: the dietary validator, the allergy checkpoint and the nutrition engine.

If the model's output cannot be used, the backend falls back to a retrieved dataset recipe, then to a template. Every response is tagged with its source, so the user never gets an empty answer. The Diet planner is a separate module. It reuses the same diet rules and nutrition engine on a dedicated database of 46,514 plan-ready recipes.

4 Data

4.1 Recipe dataset for fine-tuning

The model learns from Food.com `RAW_recipes.csv` (Majumder et al., 2019): 231,637 recipes with ingredients, steps, tags, nutrition and cooking time. The data was processed in four steps:

1. **Clean** (`preprocess_dataset.py`): drop invalid records, de-duplicate and normalise ingredient names, leaving **229,168** usable recipes.
2. **Label by diet** (`classify_diet.py`): Non-vegetarian 41.7%, Eggitarian 21.1%, Vegetarian 20.8%, Vegan 16.4%. The labelling uses the same rules as the runtime validator.
3. **Build instructions** (`create_instruction_dataset.py`): turn recipes into prompt and response pairs from 5 templates, such as ingredients to recipe, diet-constrained recipe and cuisine-specific dish. Each response is a JSON recipe.
4. **Split** 80/10/10. Training used 6,000 balanced examples and 200 validation examples, with the test split held out.

4.2 Diet planning dataset

No single public dataset has real instructions, ingredient quantities, gram-based nutrition including fibre, and photos. `build_diet_dataset.py` therefore joins two sources:

- the 2020 Food.com crawl (490,457 parsed rows), which supplies quantities, servings, nutrition and photos;
- `RAW_recipes.csv`, matched by recipe id, which supplies the instructions. Joins whose names disagree are rejected.

Rows are then filtered. A recipe is kept only if:

- it has a photo and at least 2 steps;
- it has 50 to 1,300 kcal per serving;
- its macros agree with its calories (Atwater 4/4/9 ± 25%);
- it contains no alcohol and is a meal rather than a dessert or sauce.

Diet is classified from ingredients, never from the title. Uncertain ingredients, such as Worcestershire sauce or unspecified broth, are flagged as *needs verification* and kept out of plans for the diets they

affect.

Diet	Recipes	Cuisine	Recipes
Vegan	3,822	Indian	1,140
Vegetarian	6,972	Italian	4,489
Eggetarian	5,476	Continental	11,688
Non-vegetarian	30,244	**Total plan-ready**	**46,514**

A lookup table also holds per-serving nutrition and a photo for all 269,110 Food.com ids, so the chatbot can attach nutrition to any recipe it retrieves. The USDA FoodData Central SR Legacy table holds 7,805 foods, plus 12 supplementary records for Indian and sports staples such as paneer, dals and soya chunks.

5 Model and fine-tuning

The base model is **Qwen/Qwen3-4B**, an open-weight model with 4 billion parameters. Its weights stay frozen and 4-bit quantised (QLoRA, Dettmers et al., 2023). LoRA (Hu et al., 2022) trains a small low-rank update, under 1% of the parameters, saved as a 132 MB adapter.

Setting	Value
Method	QLoRA via Hugging Face PEFT, TRL and Transformers
Rank / alpha / dropout	r = 16, α = 32, 0.05
Target modules	q, k, v, o, gate, up and down projections (all 7)
Data	6,000 examples, 1 epoch, 375 optimiser steps
Batch	1 per device × 16 gradient accumulation = 16
Learning rate / max length	2e-4 / 640 tokens
Hardware / time	NVIDIA RTX 4070, 55.4 minutes
Loss	train 1.191, best validation 1.145

Training loss fell from about 2.0 to 1.2 within the first 40 steps and then flattened. The full workflow is in `notebooks/qwen3_4b_finetuningFinal.ipynb` and `scripts/train_qwen3_4b_lora.py`.

6 Dietary safety layer

The model suggests; a rule-based validator decides. It maps each ingredient to a group (meat, poultry, fish, seafood, egg, dairy or plant) and checks the group against the preference:

Preference	Meat, poultry, fish, seafood	Egg	Dairy	Plant
Vegan	✗	✗	✗	✓
Vegetarian	✗	✗	✓	✓
Eggitarian	✗	✓	✓	✓
Non-vegetarian	✓	✓	✓	✓

An ingredient that breaks the rule is swapped for a compliant one. The quantity is kept and the steps are rewritten to match; for example, ghee becomes vegetable or coconut oil, and honey becomes maple syrup in a vegan dish. Matching is whole-word and longest-first, so almond milk, coconut cream, peanut butter and eggplant are not flagged. The validator also re-checks every recipe in the diet database at runtime. Recipes whose re-classified diet is not allowed are rejected; for example, plain feta is no longer treated as vegan.

7 Allergy checkpoint

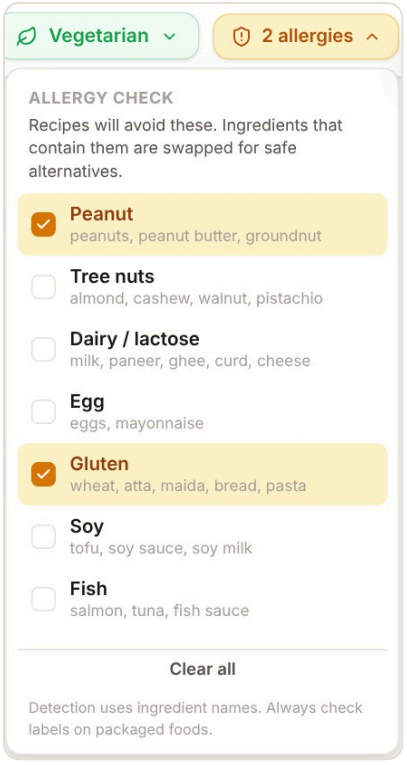


Figure 2. Allergy menu in the chat header.

Users pick allergies from 10 options: peanut, tree nuts, dairy, egg, gluten, soy, fish, shellfish, sesame and mustard. The list is saved to their profile (`PUT /api/user/allergies`) and sent with every chat. `allergy_check.py` then processes each recipe:

- Every ingredient that contains a selected allergen is replaced with the **first alternative that does not bring in another of the user's allergens**. For example, paneer becomes firm tofu, or chickpeas if the user is also allergic to soy.
- Steps and title are rewritten in a single pass with one combined pattern, so a replacement is never replaced again. "Peanut butter" becomes "sunflower seed butter", never "sunflower seed vegetable oil".
- Anything that cannot be swapped is listed as a warning, and the card states that detection is based on ingredient names.



Figure 3. Allergy check on a recipe for a user allergic to peanut and gluten: hokkien noodles → rice noodles, peanut oil → sunflower oil, kecap manis → gluten-free tamari with jaggery.

8 Nutrition engine

One engine (`nutrition_service.py`) serves the chatbot, the plans, the assistants and Check My Diet. The frontend only displays what the backend returns. Every chat recipe shows calories, protein, carbohydrate, fat and fibre per serving. Values come from one of three sources:

- **Recipe data:** recipes from the dataset use their published Food.com values per serving.
- **Approximate:** the model writes ingredient names without amounts. `portion_estimator.py` gives each ingredient the *median amount per serving* that real Food.com recipes use (1,071 ingredients, in `typical_portions.json`). These amounts are shown as suggested quantities. Nutrition is calculated from USDA FoodData Central and labelled *Approximate*. Ingredients with no USDA match are listed rather than guessed.
- **Calculated estimate:** when a plan meal is edited, the published values are adjusted by the USDA difference between the new and original ingredients. Quantities are converted to grams with USDA portion weights.



Figure 4. Nutrition per serving on a model-written recipe, labelled Approximate.

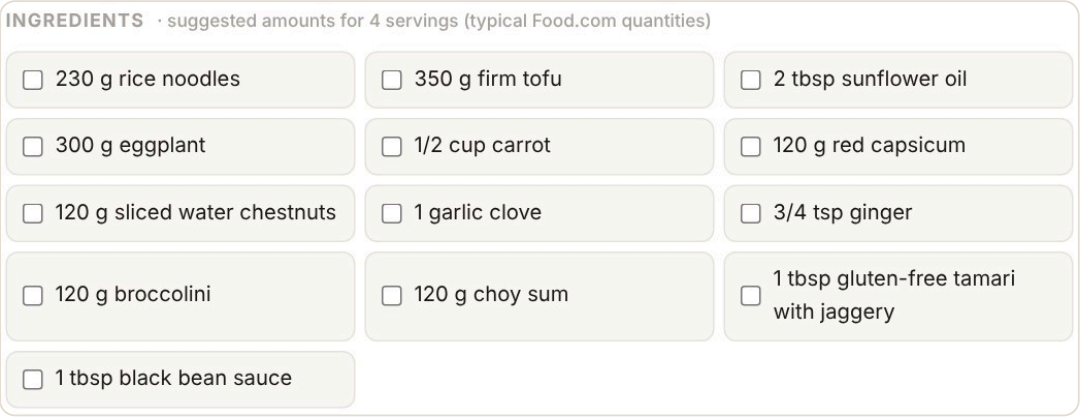


Figure 5. Suggested amounts for 4 servings, from typical Food.com quantities.

9 Diet planning

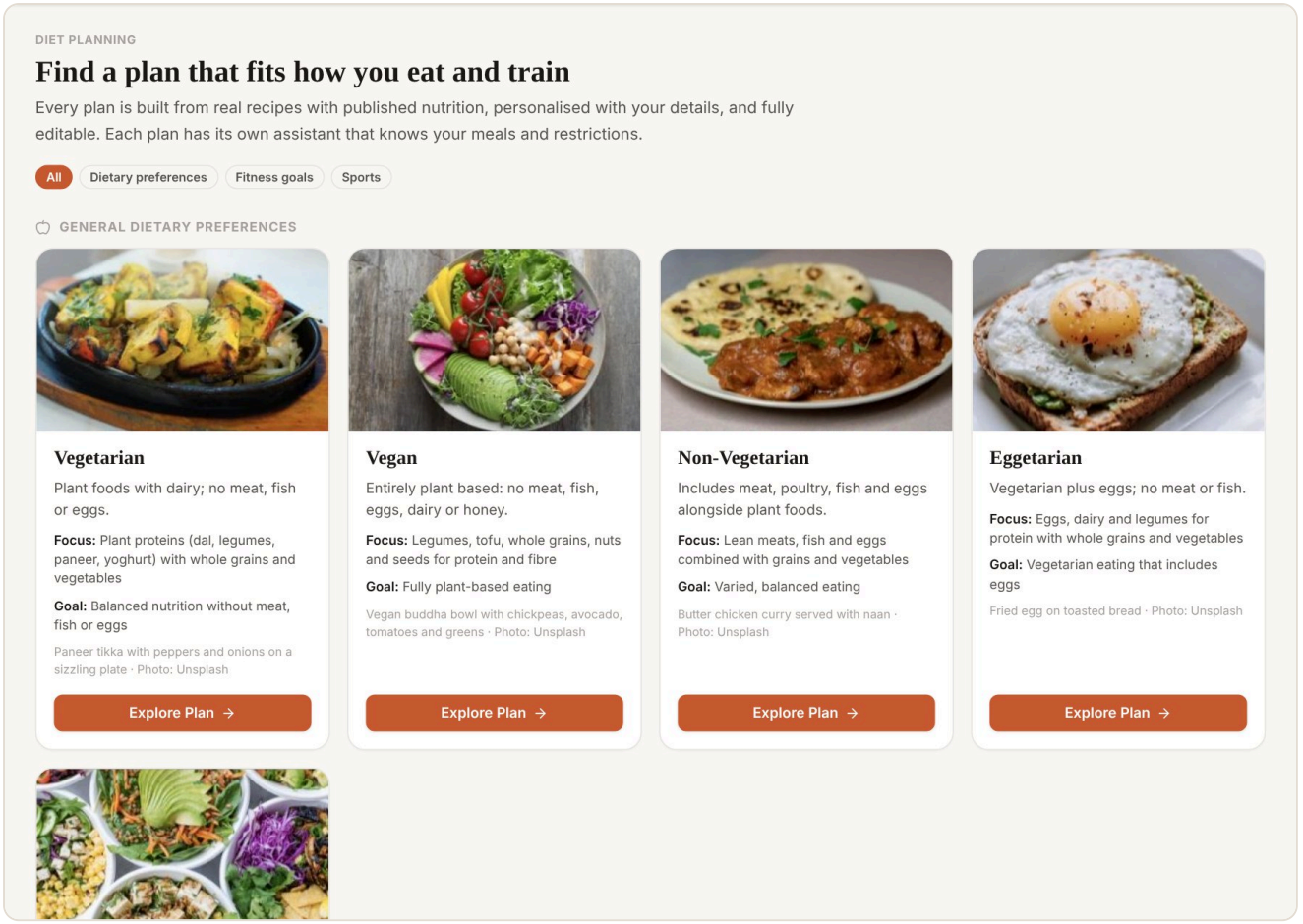


Figure 6. Diet dashboard. Each of the 24 cards has a food photo, focus and goal.

9.1 Plans and targets

The 24 plans fall into three groups:

- **Dietary preferences (5):** Vegetarian, Vegan, Non-vegetarian, Eggetarian, Balanced.
- **Fitness goals (7):** Weight loss, Weight gain, Muscle building, High-protein, Low-carbohydrate, Endurance, Recovery.
- **Sports (12):** Cricket, Football, Basketball, Tennis, Badminton, Swimming, Running, Cycling, Gym, Bodybuilding, Yoga, Other sports.

Targets use only what the user enters:

- **Energy:** Mifflin-St Jeor × activity level, adjusted by goal.
- **Protein:** 0.8 to 2.2 g/kg by goal (ACSM 2016, ISSN 2017).
- **Carbohydrate:** 3 to 12 g/kg on training days by training load (ACSM 2016).
- **Fat:** 20 to 35% of energy.
- **Fibre:** 14 g per 1,000 kcal.

Without age, height and weight there is no energy target, and the review says so.

9.2 Plan generation and editing

Meal slots are placed around training: a pre-training meal or snack, and a recovery meal after the session. SQL hard filters remove recipes that break the diet, allergies, intolerances, dislikes, cuisine or maximum cooking time. A greedy optimiser then picks recipes and portions so that energy and macros fall inside the user's ranges.

Every meal and ingredient can be replaced, added, removed or resized. Each edit is validated on the server, saved with a version check, and followed by a recalculation of totals and a fresh review.

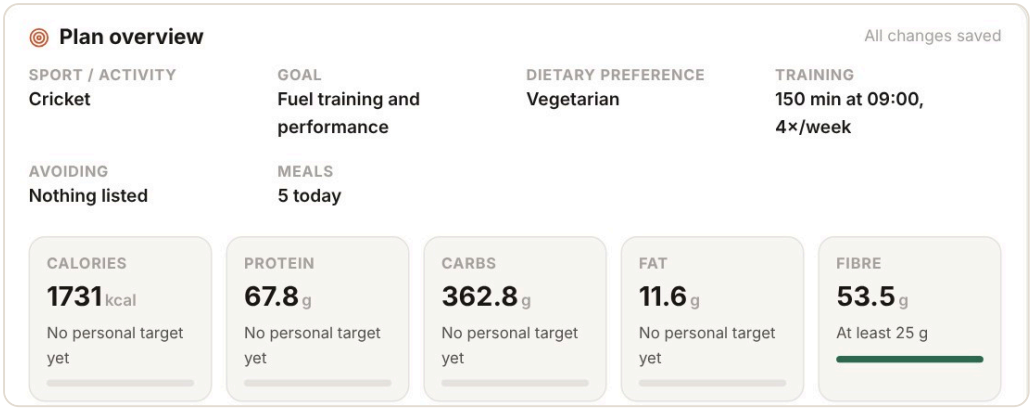


Figure 7. Plan overview with daily totals.

9.3 Check My Diet

Check My Diet compares the plan with the user's targets. It covers:

- calories, macros and fibre;
- protein spread across meals;
- pre- and post-training meals;
- diet, allergen and dislike conflicts;
- missing data.

Each finding is typed as a *restriction*, a *nutrient gap* or a *data / timing* issue. The plan gets one overall label rather than a score: Well aligned, Minor adjustments, Significant gaps or More information needed.

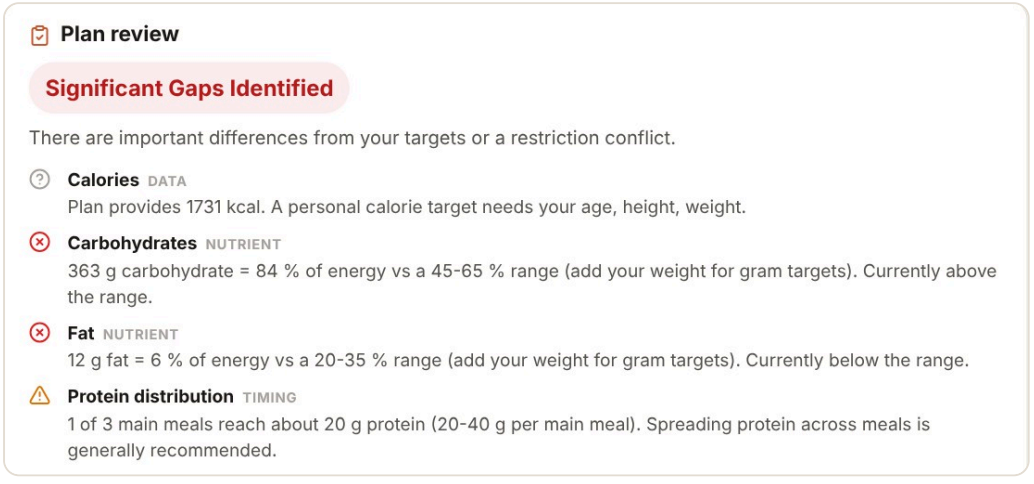


Figure 8. Check My Diet review for a Cricket plan.

9.4 Plan-specific assistant

Each plan has its own conversation, stored per plan and visible only to its owner. Deterministic handlers answer common requests from the plan data, for example:

- why a meal is included and how much protein the day gives;
- ingredient alternatives, and a lighter or higher-protein meal;
- pre-training snacks, the night before a match, rest days and hydration.

A requested change comes back as a **proposal** with a preview of the new totals and review label. The plan changes only when the user presses Apply.

Free-form questions reuse the same Qwen3-4B with the recipe adapter temporarily disabled. Any sentence that states calorie or gram figures is removed, so the model cannot invent nutrition values.

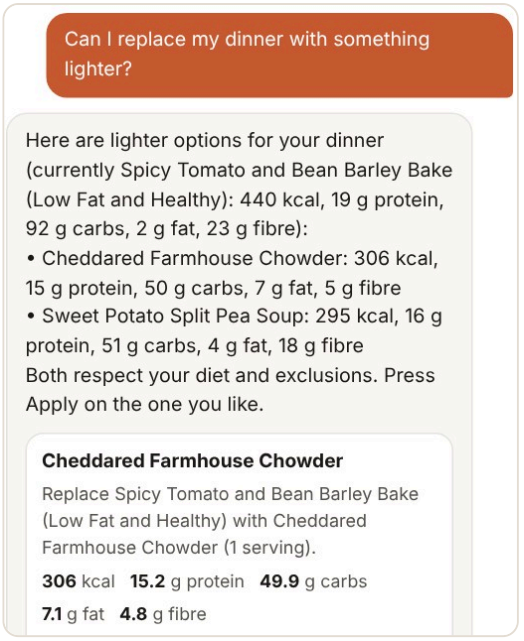


Figure 9. The assistant proposing lighter dinners that respect the plan's diet.

10 Backend and frontend

Backend. The API is a Python 3.12 FastAPI app served by Uvicorn, with these components:

- accounts: JWT tokens (python-jose) and bcrypt password hashing;
- storage: SQLite by default or PostgreSQL, through SQLAlchemy;
- inference: PyTorch, Transformers and PEFT load the base model and adapter once at startup.

The JWT secret is read from the environment, and a random one is generated if it is missing. Message length is capped at 1,000 characters, and the direct recipe endpoints require sign-in.

Endpoint	Purpose
`POST /api/auth/register`, `/login`	Create account, sign in, return token
`GET/PUT /api/user/preferences`, `/allergies`	Diet preference and allergy list
`POST /api/chat`	Recipe with diet check, allergy check and nutrition
`GET /api/chat/history`, `/api/chat/{id}`	List and open past chats
`/api/recipes/save`, `/api/favorites`	Saved recipes and favourites
`GET /api/diet/categories`, `/recipes`	Diet cards, recipe search and filters
`POST /api/diet/plans`, `/plans/{id}/edit`	Generate and edit a plan
`POST /api/diet/plans/{id}/assess`	Check My Diet
`GET/POST /api/diet/plans/{id}/chat`, `…/apply`	Plan assistant and applying a proposal

Frontend. The UI is built with React 19, TypeScript and Vite, and calls the API with Axios. It has these pages:

- login, registration and onboarding;
- the chat dashboard, with the diet and allergy toggles;
- saved recipes and settings;
- the Diet dashboard and plan pages.

The sidebar becomes a slide-in drawer on screens narrower than 900 px.

11 Evaluation

The base and fine-tuned models were run on the same prompts and the same held-out test split.

Metric	Base Qwen3-4B	Fine-tuned (QLoRA)
Test loss	2.895	**1.106** (62% lower)
Perplexity	18.1	**3.0**
Format completeness (usable JSON recipe)	66.7%	**97.0%**
Dietary compliance	100%	**100%**

The validator was tested separately on 50 hand-written ingredient lists (15 vegan, 15 vegetarian, 10 eggitarian, 10 non-vegetarian). All 50 passed, with no conflicts missed, at about 0.1 ms per check.

The automated test suite has **73 passing tests** covering five areas:

- the classifier and backend;
- frontend and backend synchronisation;

- the Diet section: daily totals equal the sum of meals, plans are scoped to their owner, and edits re-run the review;
- allergy swaps, including that a swap never adds another allergen;
- approximate nutrition and the auth requirement on recipe endpoints.

A check of 222 generated meals found no diet violations after the runtime re-check. The plan assistant answered 17 of 18 typical questions from plan data without calling the model.

12 Deployment

All three options are self-hosted. Each needs about 8 GB for the base model weights, downloaded once from Hugging Face, plus the 132 MB adapter and the diet database (about 260 MB, built by `scripts/download_diet_data.sh`).

Option	Hardware	How it runs
Local Mac	Apple Silicon, 16 GB RAM	`./run_on_mac.command`: builds missing data and the frontend, starts the backend on PyTorch MPS
GPU server	NVIDIA CUDA GPU, 12 GB+ VRAM	Uvicorn with one worker behind Nginx and HTTPS (Certbot)
Docker	Any Docker host	`docker compose up --build` starts the backend and PostgreSQL

Configuration lives in `.env`:

- `SECRET_KEY`: set a fixed value in production so sign-ins survive restarts;
- `DATABASE_URL`, `CORS_ORIGINS`, `BASE_MODEL_NAME` and `FINAL_ADAPTER_DIR`;
- `SLM_GENERATE`: set it to false for a fast retrieval-only mode in which every safety check still runs.

```
SLM_GENERATE=false python -m pytest tests/test_classifier.py tests/test_backend.py \
  tests/test_synchronization.py tests/test_diet.py tests/test_allergy_nutrition.py
```

13 Limitations and future work

- **Name-based detection.** Diet and allergen checks read ingredient names, so hidden ingredients in packaged products (anchovies in Worcestershire sauce, wheat in a sauce mix) can be missed. The app always asks users to check labels; it is not medical advice.
- **Approximate nutrition.** Values for model-written recipes use typical Food.com amounts rather than the user's exact quantities.
- **Short training run.** Training covered one epoch on 6,000 examples. More of the 229,168 recipes should improve quality further.
- **Coverage.** Food.com is mostly North American, with only 1,140 Indian plan recipes. Indian-only plans sometimes fall back to other cuisines, and the plan says when this happens.
- **Latency.** The 4B model takes several seconds per recipe on a laptop. A quantised build (GGUF or MLX) or a GPU server would speed this up.

Next steps are:

- label and barcode scanning for packaged foods;
- a larger training run;
- quantised inference;
- more Indian recipes;
- a mobile app with meal reminders.

14 References

1. Qwen Team. *Qwen3 Technical Report*, 2025.
2. Hu, E. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
3. Dettmers, T. et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023.
4. Majumder, B. P. et al. *Generating Personalized Recipes from Historical User Preferences*. EMNLP 2019 (Food.com dataset).
5. U.S. Department of Agriculture. *FoodData Central, SR Legacy*, 2018.
6. Thomas, D. T., Erdman, K. A., Burke, L. M. *Nutrition and Athletic Performance*. ACSM, AND and DC joint position statement, 2016.
7. Jäger, R. et al. *ISSN Position Stand: Protein and Exercise*. JISSN, 2017.
8. Mifflin, M. D., St Jeor, S. T. et al. *A new predictive equation for resting energy expenditure*. Am J Clin Nutr, 1990.